Combinational Logic – Part 4

K. Sridharan

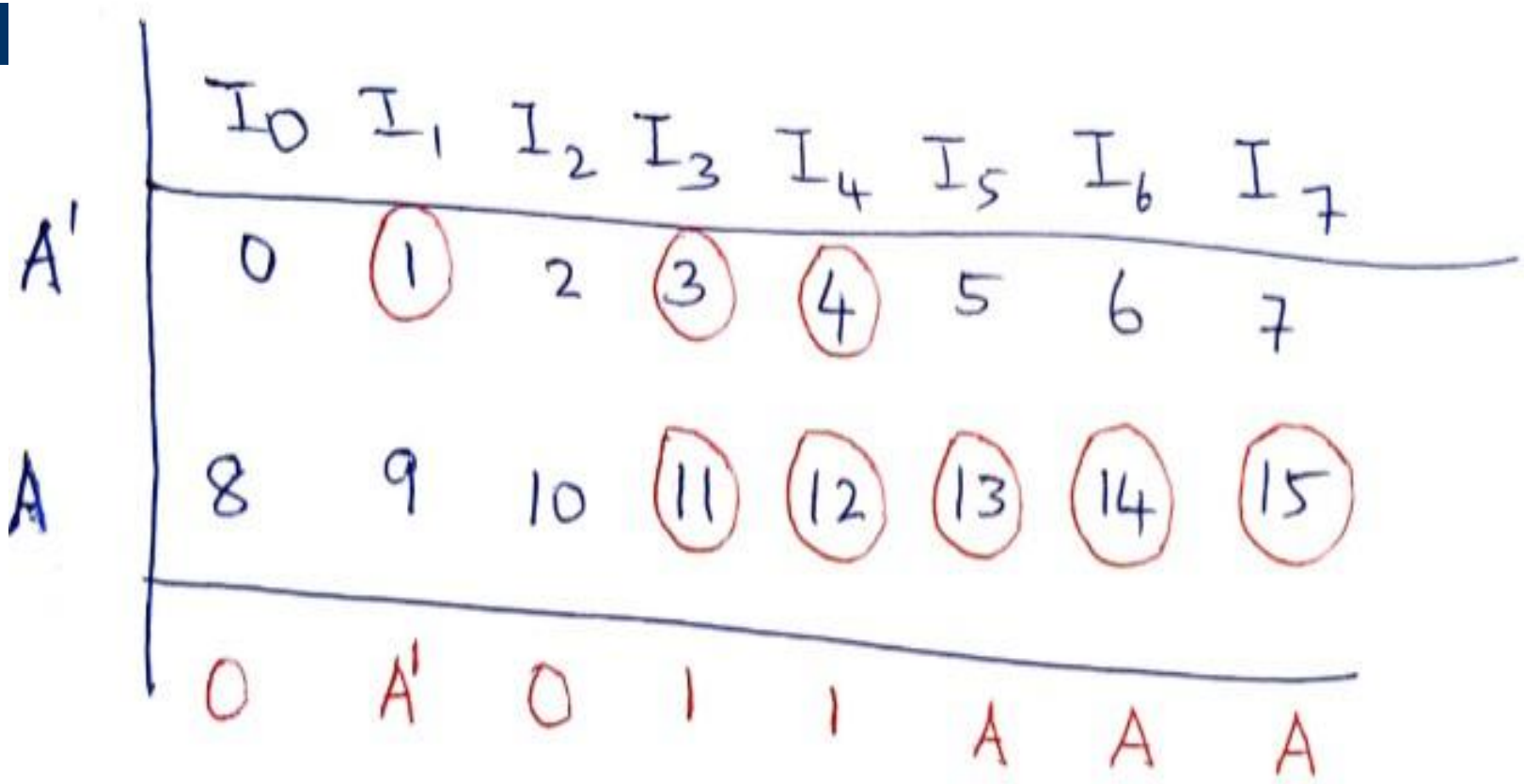
Review

- We have discussed multiplexers.
- We will now discuss some more examples for implementing a Boolean function using a Mux.

Realizing F(A,B,C,D) =∑ (1,3,4,11,12,13,14,15)

- The function here has 4 variables. We can use (n-1) variables for the selection input. So 3 selection lines and an 8-to-1 mux.
- We will let first variable B to be connected to input S_2 and the other variables (C and D) to be connected to S_1 and S_0 respectively.
- Data inputs (A, A', 0, 1) will come from the implementation table.

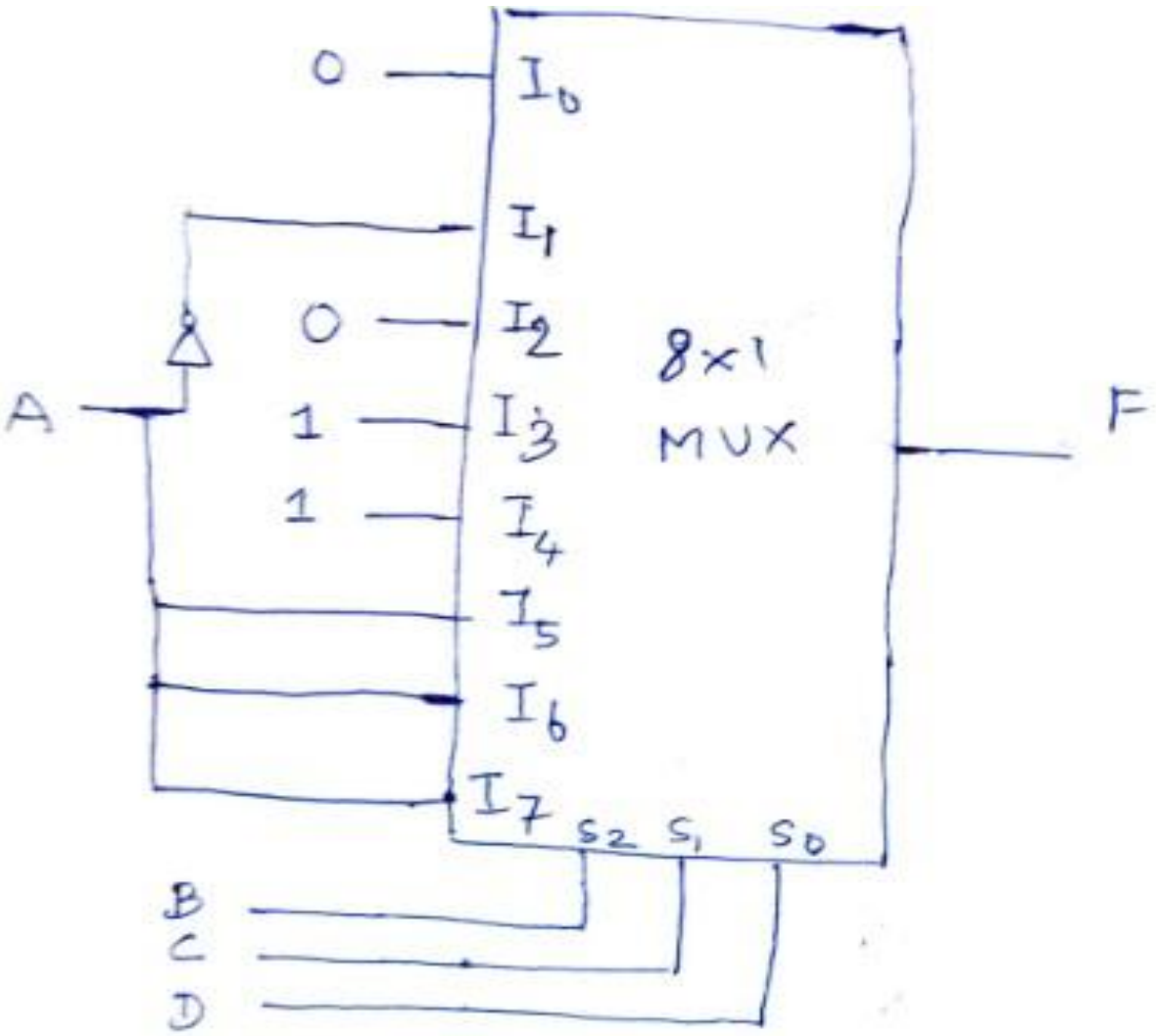
Implementation Table



On the implementation table

- We have chosen to apply the variables B, C and D to the selection lines.
- Hence, the implementation table has the first half associated with A’ (i.e., the top row) and the second half (second row) associated with A
- Circle the minterms of the function and apply the rules for finding values for the multiplexer data inputs (as for the previous example).

Mux realization



Explanation

- Once again, our Mux realization follows from the implementation table.
- Need to pay attention to the labels for the select lines.
- Once these select lines are decided, move on to the data inputs and apply as per the implementation table.

Do we need a large MUX always ?

- Suppose we want to realize

  $F(A,B,C,D) = \sum (2,4,6,9,11,13)$

- We note that we have a function of 4 variables.
- So we can use 3 lines for selection i.e., an 8 x 1 MUX.
- Do we need an 8 x 1 MUX ?
- Question for students

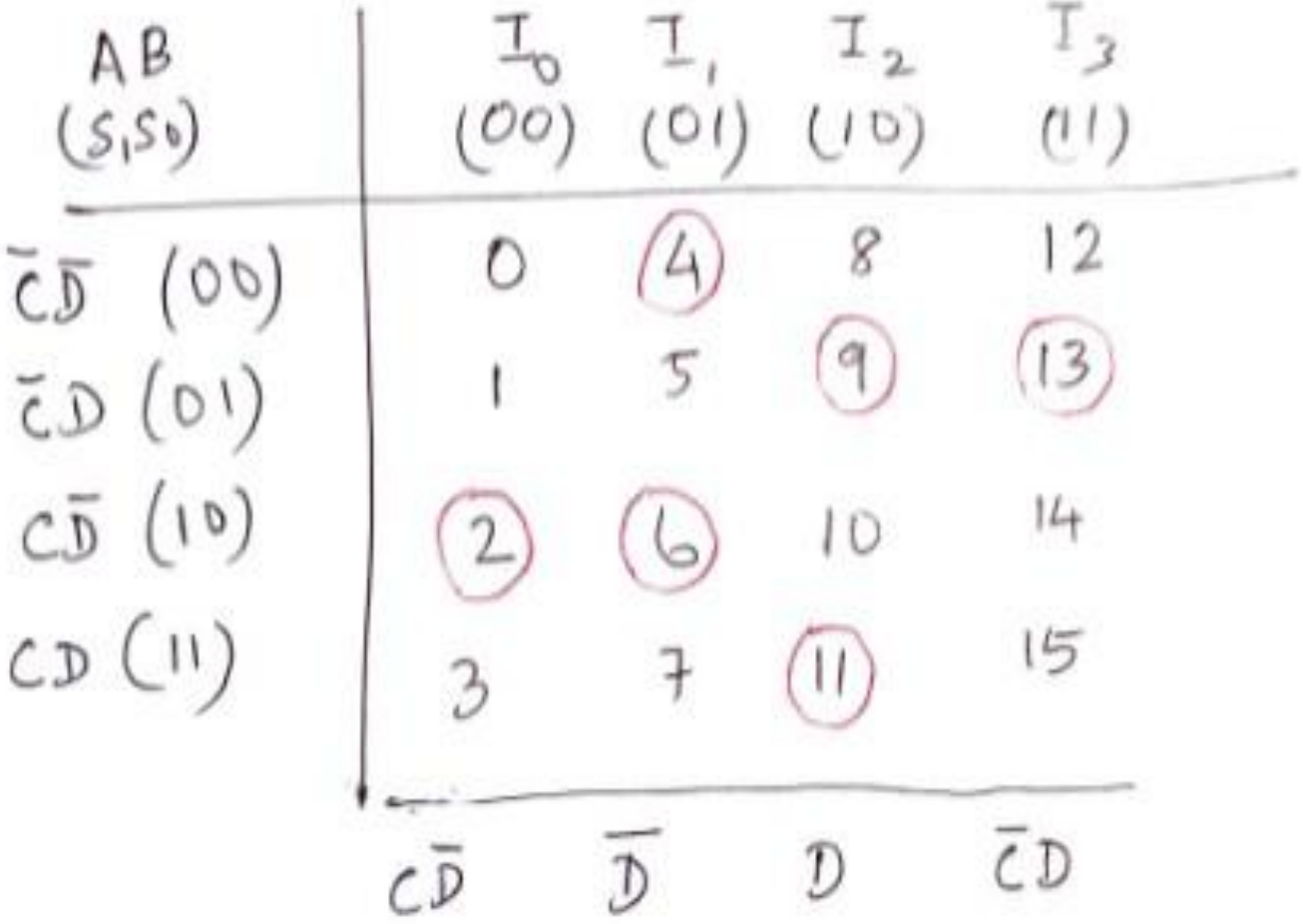
Question for students

- Suppose, I do not have an 8 x 1 MUX with me.
- I still want to use multiplexers as part of the design.
- Is there any way of doing this with a different size MUX ?

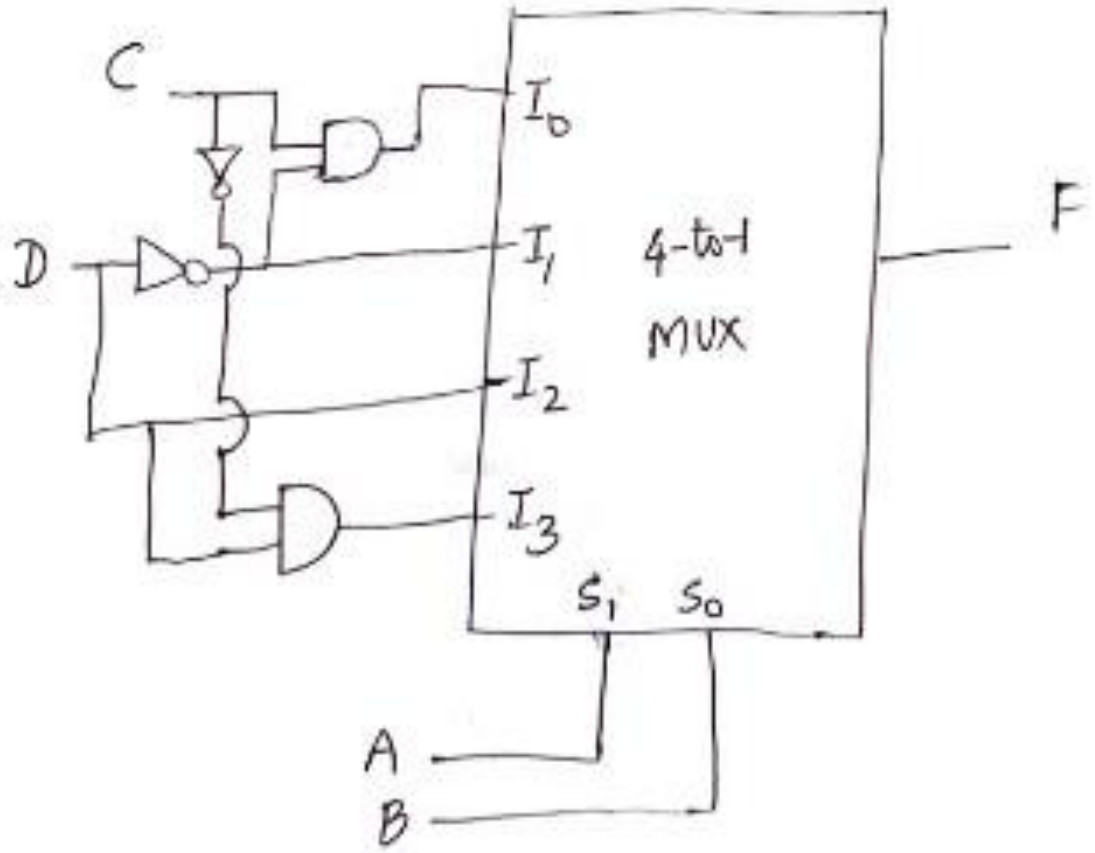
Indeed, we can use a smaller MUX ..

- We can realize this with a 4-to-1 MUX and some external gates (if required in general).
- We will show the implementation table next and follow it up with the circuit.

Implementation Table



The Mux-based circuit



Epilogue on mux …

- We have seen how a mux can be a primitive (or building block) to realize various Boolean functions.
- Compare with NAND/NOR to realize various functions.
- We will now move on to other combinational circuits.

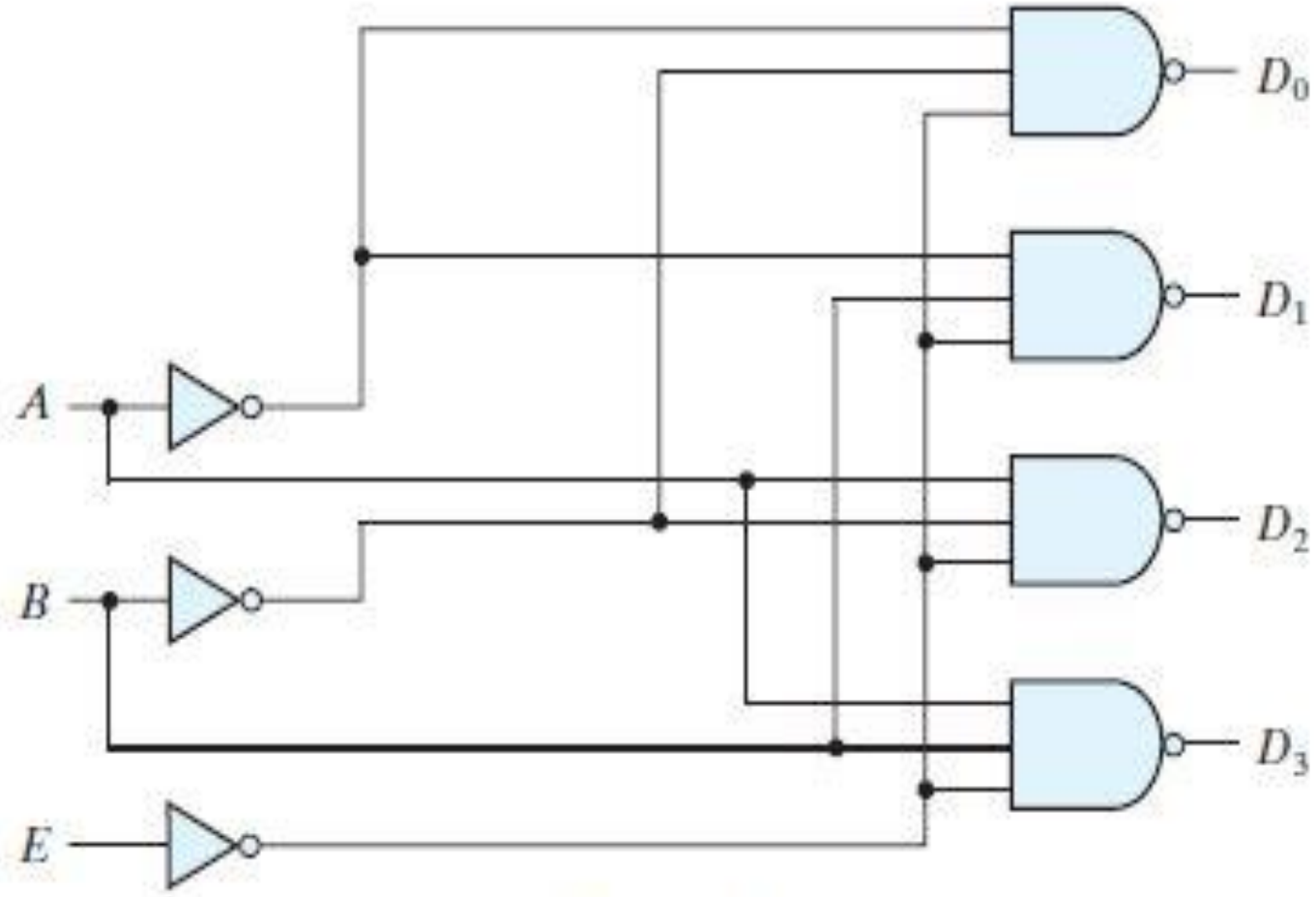
Demultiplexer

- A demultiplexer's function is inverse of that of multiplexer.
- For instance, a 1-bit, n-output demultiplexer has one data input and s inputs to select one of $n = 2^s$ data outputs.

More on Demultiplexer

- The selection of a specific output is controlled by the bit combination of the select lines.
- A decoder with an enable input works as a demultiplexer. Illustration next.

Illustration for demultiplexer



↳ active low enable

Explanation

- What we have is a 1-to-4 demultiplexer.
- E is the data input line while A and B are taken as the selection inputs.
- E has a path to all four outputs but the input information is directed to only one of the output lines (as specified by the binary combination of the two selection lines A and B).

Comparator

- A circuit that compares two binary numbers and indicates whether they are equal is a comparator.
- We can also extend this notion to indicate an arithmetic relationship (greater or less than) between a pair of binary numbers. This leads to what is generally called as a magnitude comparator

More on comparators

- Implementing the comparison of 1-bit numbers is straightforward: An XOR gate, for instance, is adequate to compare two 1-bit numbers.
- When the bits are the same, the output is zero.
- Otherwise, a 2-input XOR gate outputs a 1.

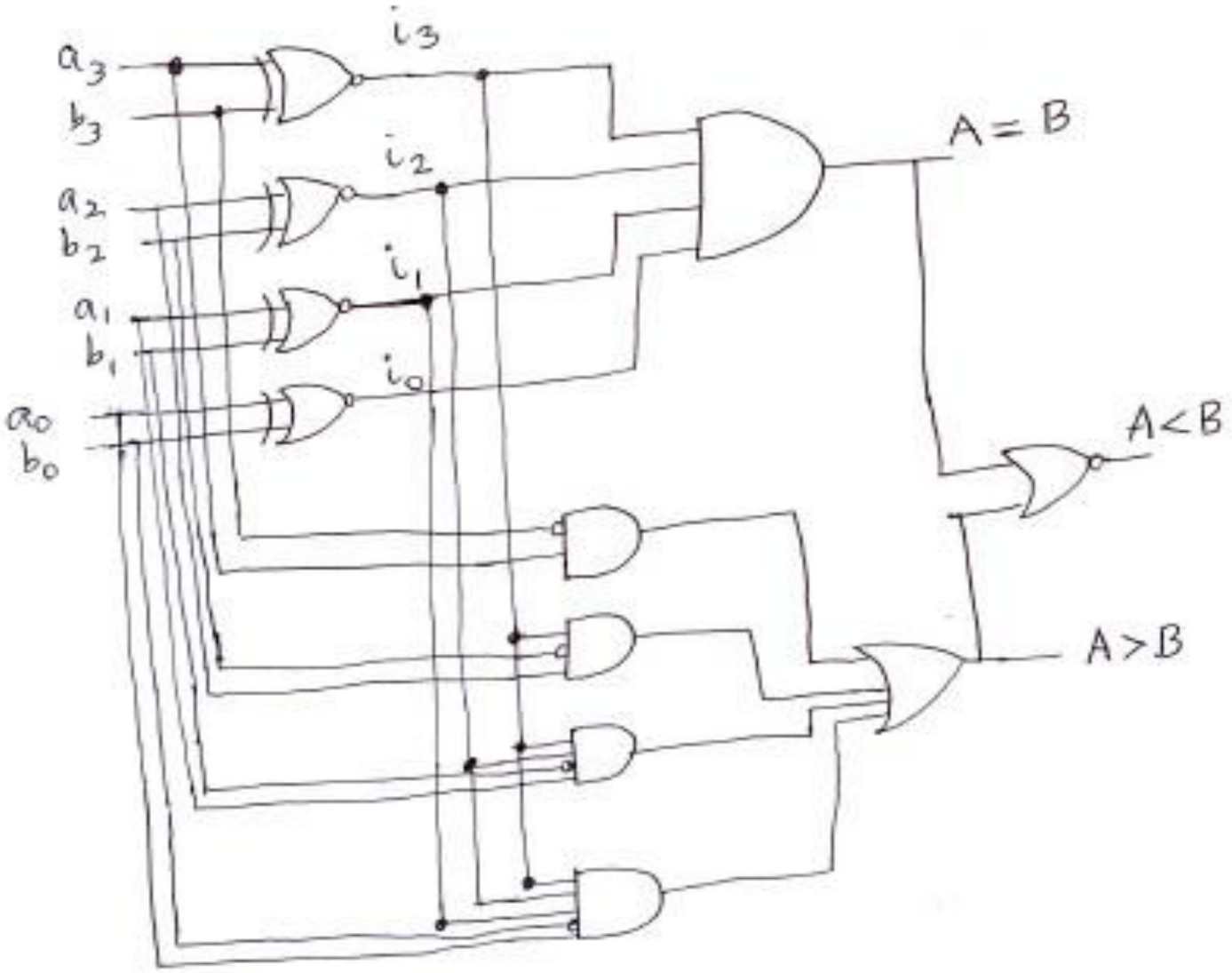
Comparing multibit unsigned numbers

- Comparing two multibit numbers (A, B) can use some of the ideas for 1-bit number comparison (the XNOR gate is useful for instance).
- However, we would like to not only check if

  A = B but also say if A > B or A < B

  (whenever A is not equal to B).
- To this end, we need a few other gates. We will show next the circuit for comparing two 4-bit numbers and then discuss how it works.

Comparing two 4-bit numbers



Explanation

- We assume A = a_3 a_2 a_1 a_0 and B = b_3 b_2 b_1 b_0 .
- We have also shown some intermediate signals, namely i_3, i_2, i_1 and i_0 .
- Note that each i_k is1 if the bits of A and B with the same index are equal. This can be expressed as i_k = (a_k xnor b_k). We can then note that A=B output corresponds to $i_3 i_2 i_1 i_0$
- Now we proceed to look at the expression that leads to A > B output. Consider bits of A and B from MSB and proceed towards LSB.

More on the comparator circuit

- If $a_k = 1$ and $b_k = 0$, then A > B. This leads to the expression

  $a_3 b_3' + i_3 a_2 b_2' + i_3 i_2 a_1 b_1' + i_3 i_2 i_1 a_0 b_0'$

- Note that $a_3 b_3' = 1$ if $a_3 = 1$ and $b_3 = 0$ so we can stop if this happens. Otherwise, we proceed to the next position from MSB side: note then that $i_3 = 1$ so we need to check if $a_2 = 1$ and $b_2 = 0$ which will set the second term to 1 and lead to A > B. If this too does not succeed, we proceed to see if $i_3 i_2 a_1 b_1' = 1$ and so on until the LSB position.

What if A is not equal to B (and A is not greater than B either) ?

- This means A < B and needs to be indicated via a circuit (or a suitable gate). This is what we do via the final NOR gate which takes the output of (A > B) and (A = B) as inputs.
- Clearly, (A < B) will turn out to be 1 when

  (A = B) is not true (i.e., this output is 0) and

  when (A > B) is also not true

  (i.e., this output is also 0)

Parity Circuit

- We now discuss the last topic in combinational circuits, namely parity checking.
- The notion of parity is used in digital systems for error checking.
- When digital information is transmitted from one point to another, it is possible for some bits to be corrupted during transmission. A transmitted bit could be 1 but the corresponding received bit could be 0.

What exactly is parity ?

- Suppose n bits are to be transmitted. A simple error checking mechanism is via inclusion of an extra bit that is called as a parity bit (denoted by p).
- Two kinds of parity are used: even parity and odd parity. For even parity, “p” is given a value such that the total number of 1s in the (n +1) transmitted bits is even. For odd parity, “p” is such that total number of 1s is odd.
- The sender generates the “p” bit based on the n-bit data item that is to be transmitted. The receiver checks whether the parity of the received item is correct.

What about the circuits ?

- Parity generating and checking circuits can be implemented via XOR gates. For a 4-bit data item consisting of bits x_3 x_2 x_1 x_0, even parity can be generated as

  $p = x_3 \text{ xor } x_2 \text{ xor } x_1 \text{ xor } x_0$

  At the receiving end, checking is done using:

  $c = p \text{ xor } x_3 \text{ xor } x_2 \text{ xor } x_1 \text{ xor } x_0$

  If c = 0, then received item has correct parity. If c = 1, then an error has occurred.

Remarks

- This is only a simple solution (and does not solve transmission error problems completely).
- For instance, if two or an even number of bits have their values inverted during transmission, the parity of the data item will not be changed – this means the error (that is actually present) will not be detected.

Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- S. Brown and Z. Vranesic, Fundamentals of digital logic with VHDL design, McGraw Hill, Education, 2017.